
Analisi Tecnica del Firmware

Progetto *Vera-Module-RBL-XE*

Analisi del progetto a cura di RetroBit Lab

9 giugno 2026

Indice

1	Introduzione	1
2	Architettura del Sistema	1
2.1	Componenti Chiave	1
2.2	Modalità operative (BUS_MODE)	1
2.3	Ambienti di build PlatformIO	2
2.4	Interfacciamento Bus	2
3	Mapping dei Pin (ESP32-S3FN8, QFN56)	2
4	Analisi delle Tempistiche	3
4.1	Accesso diretto ai registri GPIO	4
4.2	Diagramma Temporale di un Ciclo di Lettura (PBI)	4
5	Frammento di Codice Sorgente	5
6	Diagramma di Flusso del Firmware	6
7	Conclusione	7

1 Introduzione

Questo documento presenta un'analisi tecnica del firmware per il progetto *Vera-Module-RBL-XE*. L'obiettivo del firmware, eseguito su un microcontrollore ESP32, è di interfacciarsi con il bus di un processore 6502 per emulare un dispositivo PBI (Parallel Bus Interface) o CCTL (Cartridge Control Line) per computer ATARI della serie XL/XE.

Il progetto supporta due MCU target e due modalità operative, per un totale di quattro varianti di compilazione distinte. L'analisi si concentra sulla fattibilità temporale (timing) dell'interazione tra la MCU moderna e la CPU vintage, illustrando la logica di funzionamento tramite diagrammi di flusso e temporali.

2 Architettura del Sistema

Il sistema si basa sull'interazione di due componenti principali.

2.1 Componenti Chiave

- **CPU 6502 (MOS/SALLY):** Il processore centrale dell'Atari, operante a una frequenza di clock di **1.7897 MHz** (NTSC).
- **MCU (selezionabile a compile-time):** Il microcontrollore moderno responsabile del monitoraggio del bus e dell'emulazione della periferica. Sono supportati due target:
 - **ESP32-S3FN8** (QFN56, 7×7 mm) — Dual-core Xtensa LX7 @ 240 MHz, 512 KB SRAM, 8 MB flash in-package Quad SPI, 45 GPIO bidirezionali.
 - **ESP32-WROOM-32 / NodeMCU DevKit V1** (ESP32-PICO-D4) — Dual-core Xtensa LX6 @ 240 MHz, 520 KB SRAM, 4 MB flash, GPIO34–39 solo input.

2.2 Modalità operative (BUS_MODE)

Il firmware supporta due modalità operative, selezionate a compile-time tramite la macro `BUS_MODE`:

- **PBI mode (BUS_MODE=0):** Il modulo si collega al connettore ECI della serie XL/XE. Il firmware decodifica la pagina `$D1xx` tramite il segnale `D1XX_N`, serve una ROM da 2 KB nell'area `$D800–$DFFF` (segnale `ROM_SEL_N` dalla 74HC138), e controlla `EXTSEL_N` per disabilitare la ROM floating-point dell'Atari quando il dispositivo viene selezionato.
- **CCTL mode (BUS_MODE=1):** Il modulo si collega alla porta cartuccia. Il firmware decodifica i 5 bit bassi dell'indirizzo nell'area `$D500–$D5FF` tramite il segnale `CCTL_N` e genera il chip-select per il chip VERA.

2.3 Ambienti di build PlatformIO

Il file `platformio.ini` definisce quattro ambienti indipendenti:

Ambiente	MCU	Modalità	Piattaforma
<code>nodemcu-pbi</code>	ESP32-WROOM-32	PBI	espressif32@6.6.0
<code>nodemcu-cctl</code>	ESP32-WROOM-32	CCTL	espressif32@6.6.0
<code>esp32s3fn8-pbi</code>	ESP32-S3FN8	PBI	espressif32@6.6.0
<code>esp32s3fn8-cctl</code>	ESP32-S3FN8	CCTL	espressif32@6.6.0

La selezione dell'MCU avviene tramite le macro `MCU_ESP32_S3` e `MCU_ESP32_CLASSIC` passate come `-D` nei `build_flags`.

2.4 Interfacciamento Bus

L'ESP32 è collegato direttamente alle seguenti linee del bus del 6502 tramite tre level-shifter bidirezionali TXS0108E (3.3 V lato MCU, 5 V lato Atari):

- **Bus Dati (D0–D7):** Bidirezionale; la direzione è controllata dal segnale `R/W_`.
- **Bus Indirizzi (A0–A10):** Solo input; utilizzati per decodificare l'area di memoria di interesse. In modalità CCTL vengono usati solo A0–A4.
- **Segnali di controllo:** `PHI2` (clock CPU), `R/W_` (lettura/scrittura), `D1XX_N` o `CCTL_N` (selezione pagina), `ROM_SEL_N` (solo PBI), `EXTSEL_N` e `DEV_SEL_N` (output verso l'Atari e verso VERA).

3 Mapping dei Pin (ESP32-S3FN8, QFN56)

I pin fisici del package QFN56 sono numerati 1–56 (più il pad GND centrale, pin 57). La tabella seguente riporta i segnali del progetto con il numero GPIO logico e il numero di pad fisico del QFN56 (da Table 2-1 del datasheet ESP32-S3 v2.2).

Segnale Atari	GPIO	Pin QFN56	IO MUX name	Note
<i>Bus dati (GPIO bank-0, bit 4–11)</i>				
D0	4	9	GPIO4	TXS0108E U1 — glitch LOW 60 µs al
D1	5	10	GPIO5	glitch LOW 60 µs
D2	6	11	GPIO6	glitch LOW 60 µs
D3	7	12	GPIO7	glitch LOW 60 µs
D4	8	13	GPIO8	glitch LOW 60 µs
D5	9	14	GPIO9	glitch LOW 60 µs
D6	10	15	GPIO10	glitch LOW 60 µs
D7	11	16	GPIO11	glitch LOW 60 µs
<i>Bus indirizzi A0–A5 (GPIO bank-1, bit 1–6, GPIO.in1.val)</i>				
A0	33	38	GPIO33	TXS0108E U2
A1	34	39	GPIO34	
A2	35	40	GPIO35	
A3	36	41	GPIO36	
A4	37	42	GPIO37	
A5	38	43	GPIO38	
<i>Bus indirizzi A6–A10 (GPIO bank-0, solo PBI)</i>				
A6	12	17	GPIO12	glitch LOW 60 µs
A7	13	18	GPIO13	glitch LOW 60 µs
A8	14	19	GPIO14	glitch LOW 60 µs
A9	15	21	XTAL_32K_P	No cristallo 32 kHz!
A10	16	22	XTAL_32K_N	No cristallo 32 kHz!
<i>Segnali di controllo</i>				
PHI2	2	7	GPIO2	input — glitch LOW 60 µs
R/W_	17	23	GPIO17	input — glitch LOW 60 µs
D1XX_N	18	24	GPIO18	input — glitch LOW+HIGH 60 µs
ROM_SEL_N	21	27	GPIO21	input, PBI only
EXTSEL_N	0	5	GPIO0	output — strapping, pull-up 10 kΩ
DEV_SEL_N	1	6	GPIO1	output verso VERA — glitch LOW 60 µs
<i>Debug</i>				
UART TX	43	49	U0TXD	Serial debug @ 115200 baud
UART RX	44	50	U0RXD	non utilizzato in firmware

Attenzione: I pin fisici 28 e 30–35 del QFN56 (GPIO26–32) sono collegati internamente alla flash Quad SPI in-package. Non devono mai essere connessi a segnali esterni nella variante FN8.

4 Analisi delle Tempistiche

Una delle principali preoccupazioni in un progetto di questo tipo è la capacità della MCU di rispondere entro la finestra temporale definita dal ciclo di clock della CPU 6502.

- **Ciclo Clock 6502 (1.7897 MHz, NTSC):** $T_{6502} = 1/(1.7897 \times 10^6) \approx 559 \text{ ns}$
- **Finestra PHI2 alto (half-cycle):** $T_{PHI2\uparrow} \approx 279 \text{ ns}$ — questa è la finestra critica entro cui l'ESP32 deve leggere l'indirizzo, decodificarlo ed eventualmente pilotare il bus dati.
- **Ciclo Clock ESP32 (240 MHz):** $T_{ESP32} = 1/(240 \times 10^6) \approx 4.17 \text{ ns}$

Il rapporto tra la finestra critica e il ciclo dell'ESP32 è:

$$\frac{T_{PHI2\uparrow}}{T_{ESP32}} = \frac{279 \text{ ns}}{4.17 \text{ ns}} \approx 67 \text{ cicli ESP32}$$

Anche considerando solo la semi-finestra di PHI2 alto (il caso peggiore), l'ESP32-S3 dispone di circa **67 cicli di clock** per completare tutta la catena: lettura di `GPIO.in` / `GPIO.in1.val`, decodifica dell'indirizzo, lookup nella LUT, pilotaggio del bus dati. Poiché tutte le funzioni critiche risiedono in IRAM (`IRAM_ATTR`) ed operano solo su registri hardware, il percorso critico è tipicamente inferiore a 20 cicli.

4.1 Accesso diretto ai registri GPIO

Per eliminare qualsiasi overhead di astrazione, il firmware usa esclusivamente accesso diretto ai registri hardware dell'ESP32:

- `GPIO.in` — legge i 32 GPIO del bank-0 in un'unica istruzione a 32 bit.
- `GPIO.in1.val` — legge i GPIO del bank-1 (GPIO32–48).
- `GPIO.out_w1ts` / `GPIO.out_w1tc` — set/clear atomico dei bit di uscita (write-1-to-set / write-1-to-clear).
- `GPIO.enable_w1ts` / `GPIO.enable_w1tc` — abilitazione/disabilitazione in uscita (tristate) per il bus dati.

4.2 Diagramma Temporale di un Ciclo di Lettura (PBI)

Il diagramma seguente illustra un tipico ciclo di lettura gestito dall'ESP32 in modalità PBI.

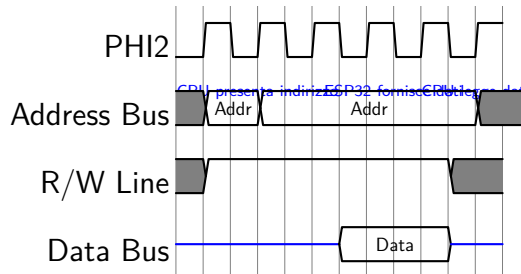


Figura 1: Diagramma temporale di un ciclo di lettura del 6502 gestito dall'ESP32 (modalità PBI).

5 Frammento di Codice Sorgente

Di seguito è riportato un estratto del file `main.cpp` che mostra la struttura di selezione a compile-time della MCU e il cuore del `MonitorTask` in modalità PBI. Tutte le funzioni critiche sono marcate `IRAM_ATTR` per garantire l'esecuzione dalla RAM interna, indipendentemente dallo stato della cache flash.

```
1 #if defined(MCU_ESP32_S3)
2     #define PIN_PHI2 2    #define PIN_RW 17
3     #define PIN_BUS_SEL_N 18    #define PIN_ROM_SEL_N 21
4     #define PIN_EXTSEL_N 0    #define PIN_DEV_SEL_N 1
5     #define PIN_UART_TX 43
6     #define DBUS_MASK \
7         ((1UL<<4)|(1UL<<5)|...|(1UL<<11))    /* GPIO4-11, 0x00000FF0 */
8     static const uint8_t DBUS_PINS[8] = {4,5,6,7,8,9,10,11};
9
10    /* A0-A5: GPIO.in1.val bits 1-6; A6-A10: GPIO.in bits 12-16 */
11    static inline uint16_t IRAM_ATTR decode_addr(uint32_t lo, uint32_t hi)
12    {
13        uint16_t a = (uint16_t)((hi >> 1) & 0x3Fu);    /* A0-A5 */
14        a |= (uint16_t)(((lo >> 12) & 1u) << 6);    /* A6 */
15        /* ... A7-A10 analoghi ... */
16        return a;
17    }
18    static inline uint8_t IRAM_ATTR decode_data(uint32_t lo) {
19        return (uint8_t)((lo >> 4) & 0xFFu); /* GPIO4-11 compatti */
20    }
21
22 #elif defined(MCU_ESP32_CLASSIC)
23     /* Pin e decode differenti per NodeMCU / PICO-D4 */
24     #define PIN_RW 15    #define PIN_BUS_SEL_N 5
25     /* D0-D7: GPIO18,19,21-23,25-27 (non contigui) */
26     static inline uint8_t IRAM_ATTR decode_data(uint32_t lo) {
27         return (uint8_t)(
28             ((lo >> 18) & 0x03u) |
29             (((lo >> 21) & 0x07u) << 2) |
30             (((lo >> 25) & 0x07u) << 5));
31     }
32 #else
33     #error "Set -D MCU_ESP32_S3 or -D MCU_ESP32_CLASSIC"
34 #endif
```

Listing 1: Selezione MCU e decode per ESP32-S3FN8 in `main.cpp`

```
1 static void IRAM_ATTR MonitorTask(void *arg)
2 {
3     bool selected = false;
4     bus_release();    /* tristate data bus */
5
6     for (;;)
7     {
8         /* 1. Attesa fronte di salita di PHI2 + snapshot */
9         uint32_t g_lo;
10        while (!(g_lo = GPIO.in) & (1UL << PIN_PHI2));
11
12        /* 2. Decodifica segnali e indirizzo */
```

```

13     bool is_read   = (g_lo & (1UL << PIN_RW))      != 0;
14     bool is_d1xx   = (g_lo & (1UL << PIN_BUS_SEL_N)) == 0;
15     bool is_rom     = (g_lo & (1UL << PIN_ROM_SEL_N)) == 0;
16     uint16_t addr  = decode_addr(g_lo, GPIO.in1.val);
17     uint8_t  off   = (uint8_t)(addr & 0xFFu);
18
19     /* 3. Selezione dispositivo (scrittura D1FF) */
20     if (!is_read && is_d1xx && off == 0xFFu) {
21         uint8_t data = decode_data(GPIO.in);
22         bool new_sel = (data == DEVICE_MASK); /* 0x80 */
23         if (new_sel != selected) {
24             selected = new_sel;
25             /* EXTSEL_N alto = ROM FP attiva; basso = disattivata */
26             if (selected) GPIO.out_w1tc = (1UL << PIN_EXTSEL_N);
27             else          GPIO.out_w1ts = (1UL << PIN_EXTSEL_N);
28         }
29     }
30     /* 4. Lettura ROM ($D800-$DFFF) */
31     if (is_read && selected && is_rom) {
32         bus_drive(pbi_driver[addr & 0x7FFu]);
33     }
34
35     /* 5. Attesa fine PHI2, rilascio bus */
36     while (GPIO.in & (1UL << PIN_PHI2));
37     bus_release();
38     GPIO.out_w1ts = (1UL << PIN_DEV_SEL_N); /* deassert VERA CS */
39 }
40 }

```

Listing 2: Nucleo del MonitorTask (modalità PBI, semplificato)

6 Diagramma di Flusso del Firmware

La logica principale del firmware è contenuta nel `MonitorTask`, che viene eseguito sul secondo core dell'ESP32 (Core 1) con priorità massima per garantire prestazioni in tempo reale.

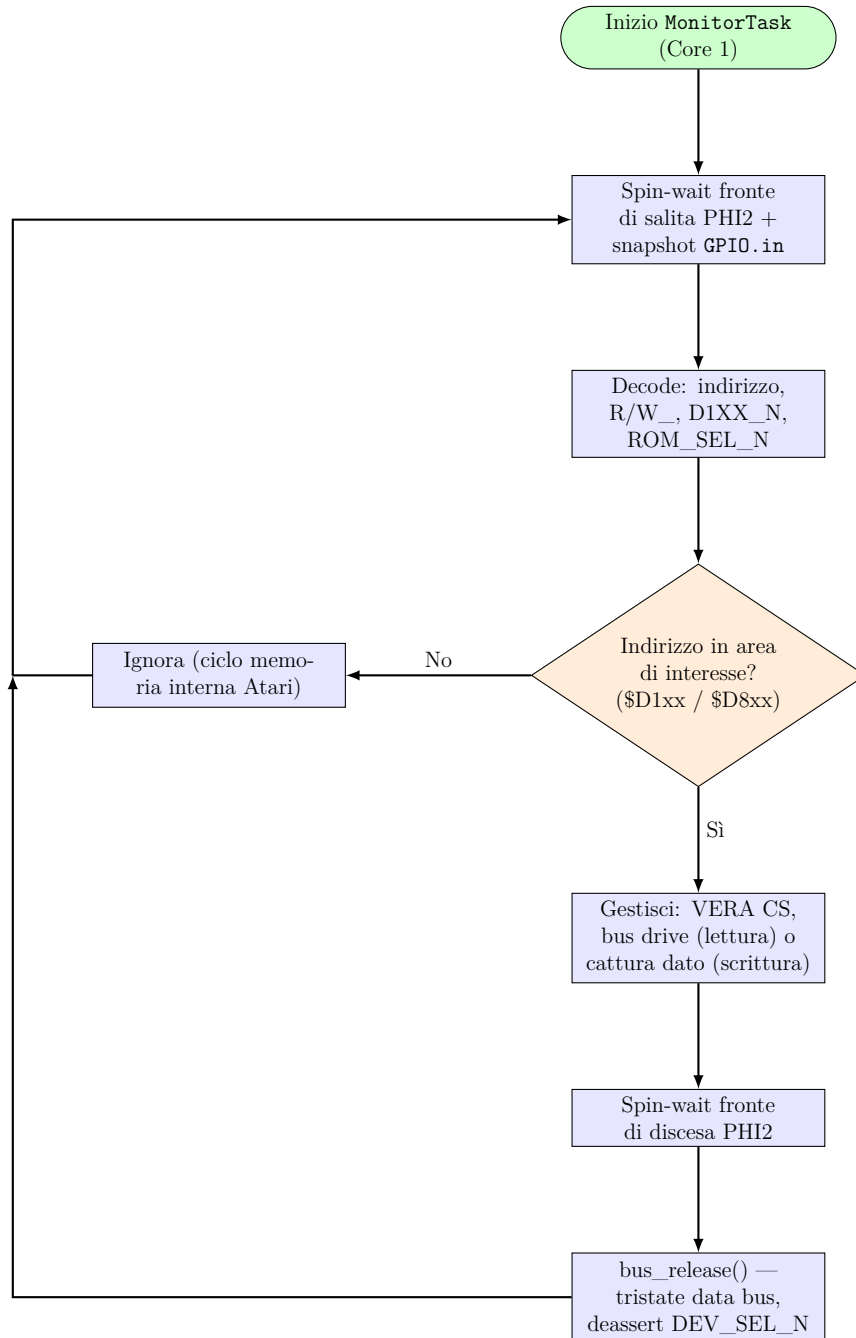


Figura 2: Diagramma di flusso semplificato del **MonitorTask** (modalità PBI).

7 Conclusione

L'analisi conferma che l'architettura hardware e software del progetto è solida e performante. L'ESP32-S3FN8 a 240 MHz garantisce circa 67 cicli di clock nella finestra critica di PHI2 alto (279 ns), ampiamente sufficienti per l'intera catena di decodifica e pilotaggio del bus. Le tecniche di programmazione a basso livello — accesso diretto ai registri GPIO (`GPIO.in`, `GPIO.in1.val`, `GPIO.out_w1ts/w1tc`), funzioni in IRAM, LUT pre-calcolata per il bus dati — garantiscono un percorso critico tipicamente inferiore a 20 cicli.

Il supporto dual-MCU (`MCU_ESP32_S3` / `MCU_ESP32_CLASSIC`) e dual-mode (`BUS_MODE=0` PBI, `BUS_MODE=1` CCTL) rende il firmware portabile su hardware diverso senza duplicazione di codice. La tabella dei pin fisici del package QFN56 consente di realizzare il PCB direttamente dalla documentazione senza ambiguità tra numero GPIO logico e pad fisico.